

Sri Lanka Institute of Information Technology



IT3012 – Intelligent Agents

Year 3, Semester 1- 2026

Practical Answer Submission

Practical Sheet No: 05

Student ID	IT24100839
Student Name	Janith Priyankara W.A.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No	Y3.S1.WE.AI.01.01

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

Answer:

Reachability checks whether a node can physically be visited. In our A* algorithm, this is checked using the grid boundaries and walls. If a tile is outside the grid or contains a wall, it is not reachable.

Feasibility is a logical check performed after the physical checks. For each possible neighboring tile, the Knowledge Base is cleared, and the relevant percepts are added as facts. The *forward_chain()* method then derives new facts using the defined rules. If Retreat is derived, the tile is considered **Infeasible** and is skipped, even though the tile may be physically reachable.

This separates physical movement constraints from logical safety constraints.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a KnowledgeBase class with a *forward_chain()* loop instead of simply writing if 'TargetVisible' in percepts and 'HasDust' in percepts: directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

Answer:

We use a *KnowledgeBase* because it separates the game rules from the agent's movement code. Instead of writing many *if* statements directly inside the A* algorithm, the rules are stored declaratively in the Knowledge Base and processed by the *forward_chain()* method.

This makes the system easier to maintain and extend. If 50 new game rules are added, we can add those rules to the Knowledge Base without creating 50 additional conditional statements inside the movement algorithm. The A* code can continue using the same inference mechanism.

3. Modus Ponens & Horn Clauses (Understand)

The rule ['TargetVisible', 'HasDust'] $\Rightarrow$ 'SafeToEngage' is formally known as a Horn Clause. Briefly explain how your Python implementation of the *forward_chain()* while loop acts as a mathematical execution of the *Modus Ponens* inference rule.

Answer:

(TargetVisible $\wedge$ HasDust $\rightarrow$ SafeToEngage) is a Horn Clause. The *forward_chain()* method implements the Modus Ponens inference process by checking whether all premises of a rule are present in the current set of facts. For example, if the Knowledge Base contains *TargetVisible*, *HasDust* the *forward_chain()* method finds that both premises are true and adds *SafeToEngage* to the facts. It continues checking the rules until no new facts can be derived. Therefore, the Python *while* loop provides an implementation of repeated logical inference using Modus Ponens.